



Document Object Model (DOM)

Manipulate the structure

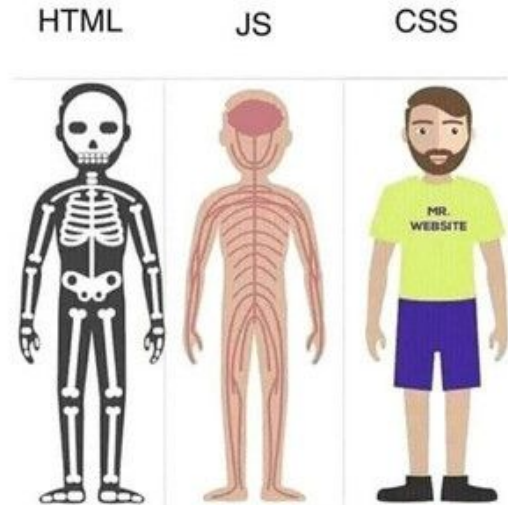
The web

Developing for the web

- **HTML**: structure
- **CSS**: layout
- **JavaScript**: interactivity in the browser



- **PHP**: interactivity on the server
- **IDE**: application to write code



Adding JavaScript

Setup

- Same HTML structure as Web 1
- CSS in the head
- `<script src="path/to/script.js">`
- Just before the closing `</body>` tag
- JavaScript at the bottom of the document to avoid [render blocking](#).

DEMO: hello-world-inline

DEMO: hello-world-external



init()

- best practice
- functions that initialize all other functions
- gives your code a clean starting point

```
const init = () => {  
  // This is the entry point of our script  
  console.log('👋 Hello from the external script!');  
}  
  
init();
```

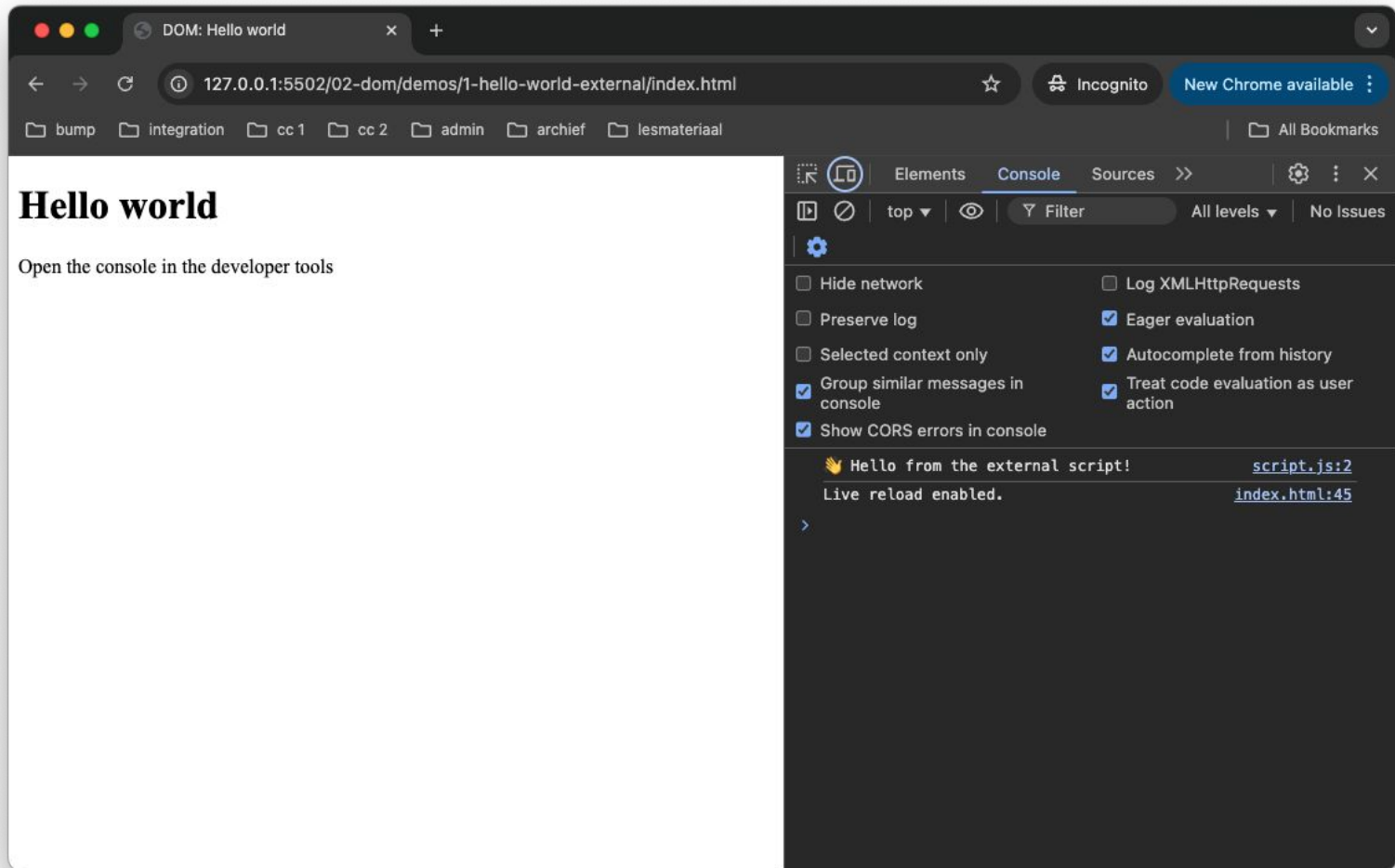


Console

- Open developer tools > console
- Use the console to watch for errors
- Use `console.log()` to test
- When your code doesn't work and nothing happens on screen, the console is the first place to check.
- Always open your console

No console open === coding blind





Arrow functions

We'll use arrow functions throughout the rest of the course, so let's learn them first

Arrow functions

A shorter way to write functions by using the fat arrow notation =>

Normal function

```
function init() {  
    console.log('👋');  
}  
  
init();
```

Arrow function

```
const init = () => {  
    console.log('👋  
Hello');  
}  
  
init();
```



Arrow functions

```
const calculateSurfaceArea =(someWidth, someLength)=> {  
    let area = someWidth * someLength;  
    return area;  
}
```

// call the function

```
const result = calculateSurfaceArea(10,6);  
console.log(result);
```



Arrow functions / parameters

No parameters

```
const doSomething = ( ) => { // Code }
```

1 parameter

```
const doSomething = param => { // Code }
```

2 or more parameters

```
const doSomething = (param1, param2 ) => { // Code }
```



Arrow functions

Single line

```
const sumNumbers = (number1, number2) => number1+number2;
```

Multi-line block

```
const sumNumbers = (number1, number2) => {  
    const sum = number1+number2;  
    return sum;  
}
```



Template Literals

Template Literals

using backtick (``) characters, we can:

- multi-line strings
- string interpolation with embedded expressions

```
`string text`
```

```
`string text line 1  
string text line 2`
```

```
`string text ${expression} string text`
```



Template Literals

```
const name = 'Sarah';  
const age = 22;
```

// String concatenation with +

```
const intro = 'Hello, I am ' + name + 'and I am ' + age + ' years old.';
```

// Template literals with backticks.

```
const intro = `Hello, I am ${name} and I am ${age} years old year.`;
```



- 1. init*
- 2. refactor to arrows*

Excercise Time

The DOM | Document Object Model

Remember last week? The DOM is object-oriented. document is an object. it has methods and properties that we will use. We are using OOP!

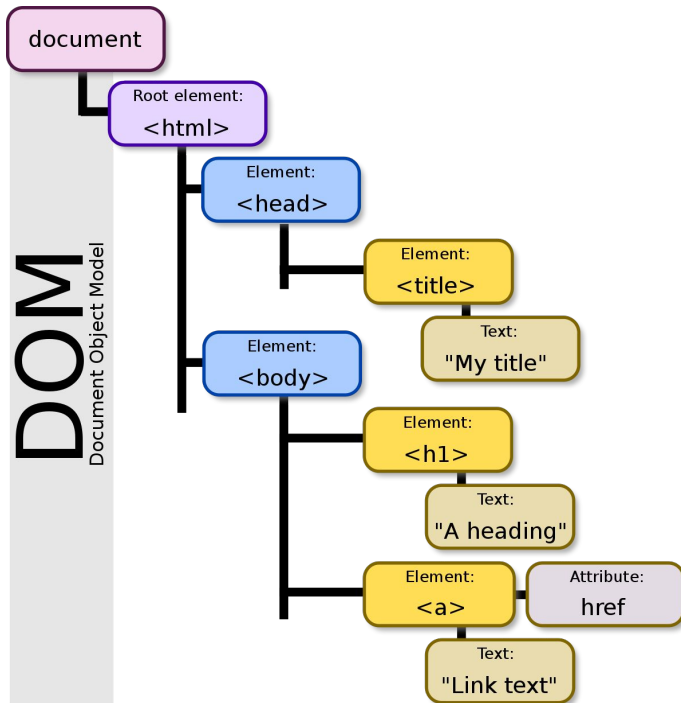
The DOM

- **Document Object Model**
- Programming interface for structured documents (like HTML)
- Connects HTML with script
- Manipulate the structure
- Available for all programming languages
- Notice that the DOM is itself object-oriented
 - document is an object
 - we call methods on it
- [MDN: Introduction to the DOM](#)



Terminology

- document: page loaded in the browser
- node: part of the document
 - document node
 - root node
 - element nodes
 - attribute nodes
 - text nodes



Select HTML Element(s)

This is how you grab a specific element to do something with it. Change its text, hide it, move it. Almost every interactive feature starts with selecting an element.

Select HTML element(s): querySelector

- Using a CSS selector
- Selects exactly 1 element
- Only the first element if more than one matches the selector
- Can be called from any element (node)
- Naming convention: variables that contain an HTML element should start with a dollar sign \$. For example:
 - \$listItem
 - \$menu
 - \$nav

DEMO: querySelector



Select HTML element(s): querySelectorAll

- Using a CSS selector
- Returns a NodeList = a list of HTML elements
 - A NodeList is not an Array!
 - forEach works on a NodeList, but map, filter, and reduce do not.
 - Convert first with Array.from() if you need any of the array functions.
- Can be called from any element (node)

DEMO: querySelectorAll



Select HTML element(s): debugging

- If `querySelector` can't find your element, it returns `null`.
- Always check your selector matches something in the HTML.
- Remember: the console is your best friend!
- Common mistakes:
 - Typo in the selector (`.titel` instead of `.title`)
 - Script runs before the HTML is loaded
 - Script is not linked in HTML.
 - Wrong selector type (forgot the `.` for class or `#` for id)



Selecting HTML element(s): FYI

- Also other function available
 - `getElementById`
 - `getElementsByClassName`
 - `getElementsByTagName`
- We do not use them in this course
- `querySelector` and `querySelectorAll` cover everything



Select HTML element(s): children & siblings

- Multiple functions available:
 - firstElementChild and lastElementChild
 - parentNode
 - nextElementSibling and previousElementSibling
 - children
- NOTE:
 - firstChild, lastChild, nextSibling en previousSibling mostly return an empty textNode (whitespace)

DEMO: children and siblings



3. select

Exercise Time!

Manipulating Elements

This is how you make your page dynamic. Updating, hiding, displaying things. All without reloading the page.

Select and manipulate content

- textContent
 - get and set text in an element
 - HTML tags and attributes are ignored
- innerHTML
 - get and set text in an element
 - get: HTML tags and attributes are received as plain text
 - set: Possible to set HTML tags and attributes

DEMO: textContent and innerHTML



Select and manipulate content

Which one to use?

- `textContent` if no further HTML is required
- Otherwise `innerHTML`

Why?

- `innerHTML` is a risk for XSS (cross-site scripting)
- `<script>` is also an HTML tag...



Select and manipulate attributes

Check MDN for the following functions. The names explain already a lot

- [getAttribute](#)
- [setAttribute](#)
- [hasAttribute](#)
- [removeAttribute](#)

DEMO: attributes



Manipulate the style: style attribute

- All elements contain a style attribute with a CSS Style Declaration object
 - Value is mostly empty
 - Is only set with inline CSS
- Sometimes you need to set a value that comes from a calculation, like a position or a width. That's when `.style` is useful. For everything else, use `classList`!

DEMO: style



Manipulate the style: classList

- Collection of all classes used on an element
- Some important functions:
 - add
 - remove
 - contains
 - toggle
- Perfect way to set style using JavaScript.
- All style rules stay in CSS

DEMO: classList



Manipulate the style: custom properties

- Update custom properties on the fly with javascript
- CSS variables have access to the DOM, which means that you can change them with JavaScript.
- [setProperty](#)
- Perfect way to dynamically style using JavaScript.

DEMO: custom properties



4. profile

Exercise Time

Creating new elements

Step 1: createElement

- Create an empty element using createElement()
 - parameter: which element
 - returns: the element itself
- \$ is a naming convention, it represents a DOM element

```
const $newItem = document.createElement('li');
```



Step 2: Set the content (manipulate styling)

- `textContent`: if content is only text
- `innerHTML`: if content also contains HTML
- `classList`: to set classes
- `setAttribute`: to set other attributes

```
$newItem.textContent = 'New list item';
```

```
$newItem.classList.add('highlight');
```



Step 3: attach to the DOM

Element must be attached to the DOM, otherwise it's not visible!

- [appendChild](#): last child of the parent
- [insertBefore](#): before another child element, also set that child element

```
const $list = document.querySelector( '.my-list' );
```

```
$list.appendChild($newItem);
```



Creating new elements: example

// Step 1: create the element

```
const $newItem = document.createElement('li');
```

// Step 2: set the content and style

```
$newItem.textContent = 'New list item';
```

```
$newItem.classList.add('highlight');
```

// Step 3: attach to the DOM

```
const $list = document.querySelector('.my-list');
```

```
$list.appendChild($newItem);
```



Create new elements

In three steps:

1. create an element
2. set the content (manipulate styling)
3. attach to the DOM

DEMO: create a new element



Random Numbers

random number between 2 values

// return a number between 0,1

```
Math.random()
```

// return a number between 0,10

```
Math.random() * 10
```

// return a number between min and max

```
Math.random() * (max-min) + min
```



Code snippet to generate random numbers

```
const randomNumber = (min, max) => {  
    return Math.floor(Math.random() * (max - min + 1) +  
min);  
}
```

DEMO: random-numbers



Data Attributes

When you need to attach custom information to an HTML element. Like an ID, a category, or some extra data.

Data Attributes

- Store custom data directly on HTML elements using data- attributes.
- [Documentation](#)

```
<article class="card" data-id="42" data-category="sport">  
    <h2>Title</h2>  
  
</article>
```



Accessing Data Attributes

- Access them in JavaScript using dataset.
- data- attribute values are always strings Use `parseInt()` or `parseFloat()` if you need numbers!

```
const $card = document.querySelector('.card');
```

```
console.log($card.dataset.id);           // "42"  
console.log($card.dataset.category);     // "sport"
```



Updating Data Attributes

```
const $card = document.querySelector('.card');
```

```
// writing
```

```
$card.dataset.status = 'active';
```

```
// → adds data-status="active" to the element
```



- 5. track list*
- 6. book list*
- 7. image grid*
- 8. saving ryan*

Exercise Time